

Київський національний університет імені Тараса Шевченка
Факультет комп'ютерних наук та кібернетики
Інструментальні засоби розробки програмного забезпечення

Лабораторна робота №1
“Практичне застосування системи контролю версій Git та юніт-тестування
у процесі розробки програмного забезпечення.”
Виконала студентка другого курсу
Групи ІПС-22
Сербін Марія Сергіївна

Київ – 2025

Тема: Практичне застосування системи контролю версій Git та юніт-тестування у процесі розробки програмного забезпечення. Формування повного робочого циклу з GitHub — від створення репозиторію до Pull Request

Мета: Отримати практичні навички використання сучасних інструментів розробки ПЗ такі, як робота з системою контролю версій Git та сервісом GitHub, побудова історії комітів і гілок проєкту, написання юніт-тестів до існуючого коду, формування правильного процесу створення Pull Request та командної взаємодії у GitHub.

Теоретичні відомості.

Git і GitHub.

Система Контролю Версій — це програмне забезпечення, що реєструє зміни у файлі або наборі файлів з плином часу. Це дозволяє користувачам відновлювати попередні версії, порівнювати зміни та відстежувати, хто і коли вніс модифікації. Основна мета — забезпечення цілісності даних та полегшення паралельної роботи над проєктом.

Git — це децентралізована (розподілена) система контролю версій (DVCS). Її ключова архітектурна відмінність полягає в тому, що кожен розробник має повну локальну копію репозиторію, включно з усією історією змін. Це забезпечує високу продуктивність операцій (**коміти** — збереження знімків стану) та можливості для нелінійної розробки (через **гілкування** та злиття).

GitHub — це веб-сервіс, який пропонує хостинг для репозиторіїв Git. Він доповнює можливості Git, надаючи інструменти для управління проєктами, візуалізації історії та обговорення коду.

Центральним елементом спільної роботи на платформі є **Pull Request (PR)** — формальний запит на злиття (інтеграцію) змін з однієї гілки в іншу. Pull Request дозволяє команді проводити огляд коду (code review), автоматично запускати тести та обговорювати запропоновані модифікації до того, як вони будуть інтегровані в основну гілку проєкту.

Юніт-тестування

Юніт-тестування (або модульне тестування) — це фундаментальний метод тестування програмного забезпечення, який полягає в перевірці найменших ізольованих частин коду, що називаються "юнітами" або "модулями".

Юніт-тести потрібні для перевірки базових випадків (тестування типових даних) та граничних випадків (перевірка роботи з даними, що не є допустимими для коректної роботи алгоритму).

Для виконання задачі юніт-тестування використовують спеціальні модулі та фреймворки, як наприклад unittest для мови програмування Python, який був використаний у даній лабораторній роботі.

Матриці

У якості коду для написання юніт-тестів було взято програмну реалізацію деяких операцій над матрицями, яка записана у файлі matrix.py.

Даний файл включає в себе:

- Додавання та віднімання двох матриць.

Операції для матриць однакової розмірності $A = [a_{ij}]_{m \times n}$ та

$B = [b_{ij}]_{m \times n}$, результатом яких є матриця тої ж самої розмірності

$C = [c_{ij}]_{m \times n}$ де $c_{ij} = a_{ij} \pm b_{ij}$

- Множення на скаляр

Добутком матриці A на скаляр λ (лямбда) є матриця B того ж розміру, де кожен елемент b_{ij} отриманий множенням a_{ij} на λ . $B = \lambda * A$, що означає $b_{ij} = \lambda * a_{ij}$

- Множення двох матриць

Добуток $C = AB$ матриці A розміру $m \times k$ на матрицю B розміру $k \times n$ є матрицею C розміру $m \times n$. Множення можливе лише тоді, коли кількість стовпців першої матриці (k) дорівнює кількості рядків другої (k). Кожен елемент c_{ij} результуючої матриці обчислюється як скалярний добуток i -го рядка матриці A на j -й стовпець матриці B : $c_{ij} = \text{Сума } (a_{ip} * b_{pj})$ для p від 1 до k .

- Транспонування матриці

Транспонованою матрицею A^T до матриці A розміру $m \times n$ називається матриця A^T розміру $n \times m$, отримана заміною рядків матриці A на її стовпці (i навпаки).

- Елементарні лінійні перетворення над рядками матриці

Множення рядка на скаляр — в i -му рядку матриці $A = [a_{ij}]_{m \times n}$ кожен елемент a_{ij} множиться на скаляр λ .

Додавання двох рядків — до кожного елементу i -того рядка матриці $A = [a_{ij}]_{m \times n}$ додається елемент j -того рядка, що стоїть в тому ж стовпчику.

- Обчислення визначника матриці.

Визначник матриці — функція від квадратних матриць, яка набуває скалярних значень. Позначення: $\det(A)$, $\det A$, $|A|$.

$$\sum_{\sigma \in S_n} \text{sign}(\sigma) a_{1,\sigma(1)} \cdot a_{2,\sigma(2)} \cdot \dots \cdot a_{n,\sigma(n)}.$$

- Обчислення матриці, оберненої до заданої

Обернена матриця A^{-1} – матриця, яка існує для кожної квадратної невинродженої матриці A , розмірності $n \times n$, причому

$$AA^{-1} = A^{-1}A = I_n$$

де I_n - одинична матриця порядку n . Якщо для матриці існує обернена, то ця матриця називається оборотною. Кожна невинроджена матриця є оборотною. Обернене твердження також є правильним.

- Обчислення рангу матриці

Ранг матриці – максимальна кількість її лінійно незалежних стовпців (або рядків).

Репозиторій

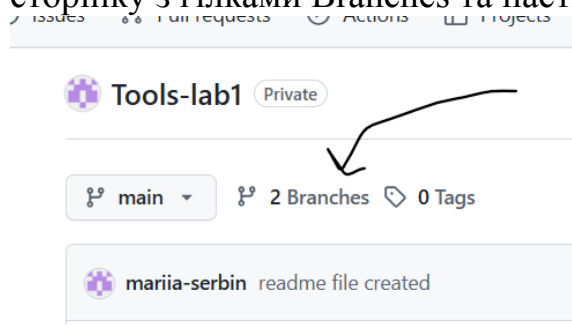
Лабораторна робота виконувалась у репозиторії GitHub, який можна знайти за [посиланням](#).

Робота з системами контролю версій (GitHub).

У цьому розділі продемонстрована робота з системами контролю версій. Демонстрація основних етапів роботи відбувається у вигляді скріншотів.

Створення гілки

Створення гілки можна виконати як і в веб-версії GitHub (іл. 3), так і в IDE (у дані лабораторній робота відбується за допомогою Pycharm)(іл.4). Для створення гілки через веб-версію потрібно у репозиторії перейти на сторінку з гілками Branches та натиснути на кнопку New Branch.

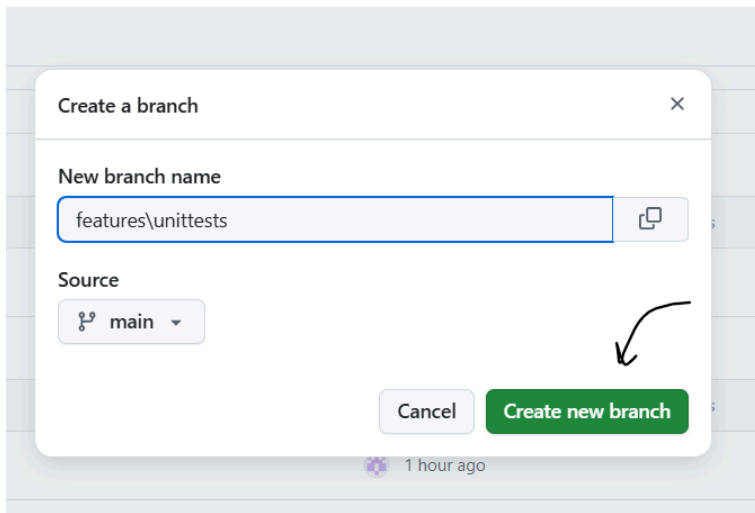


іл. 1



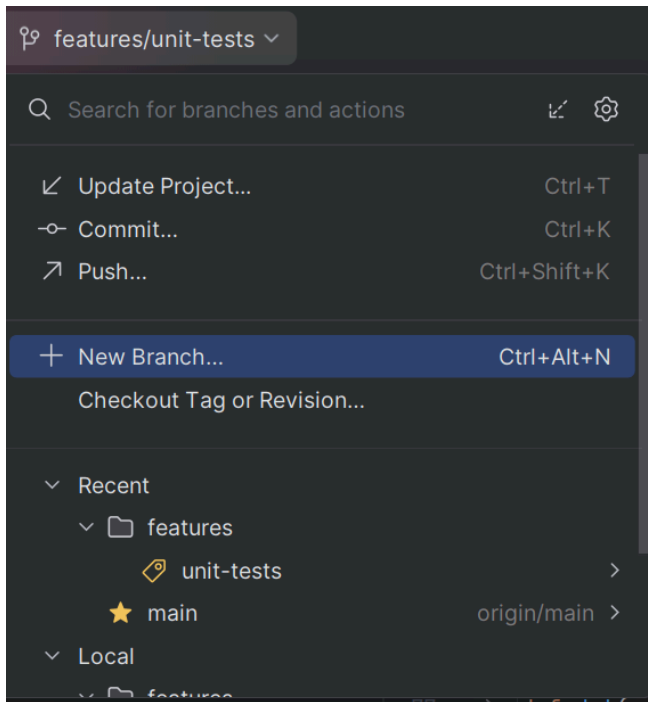
New branch

іл.2



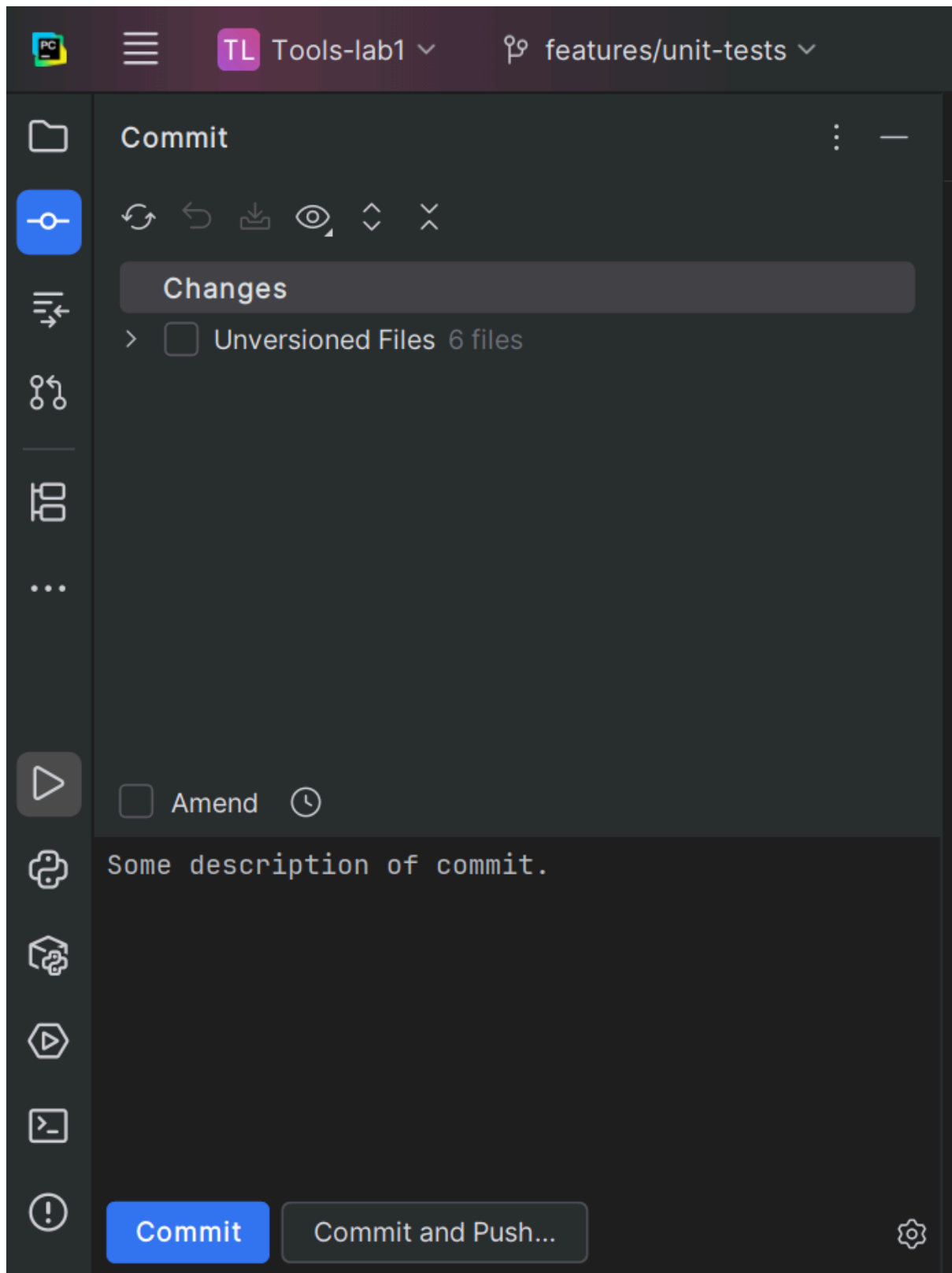
іл. 3

Створення гілки в середовищі редагування.

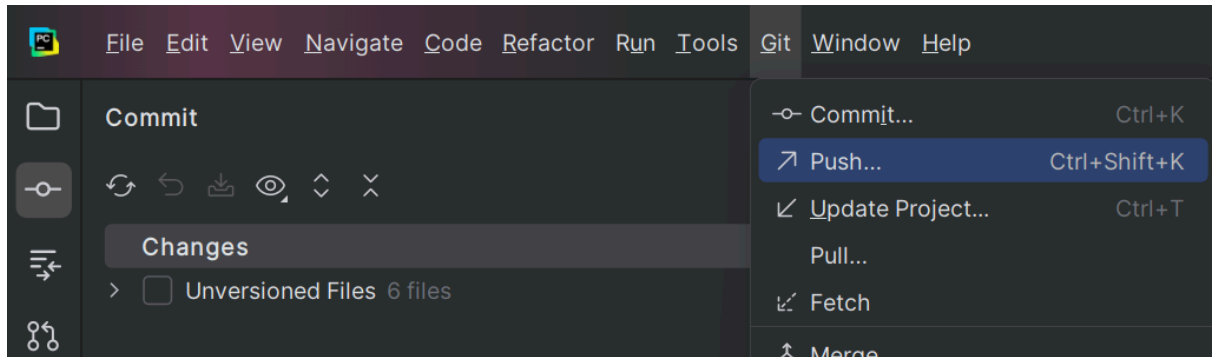


іл. 4

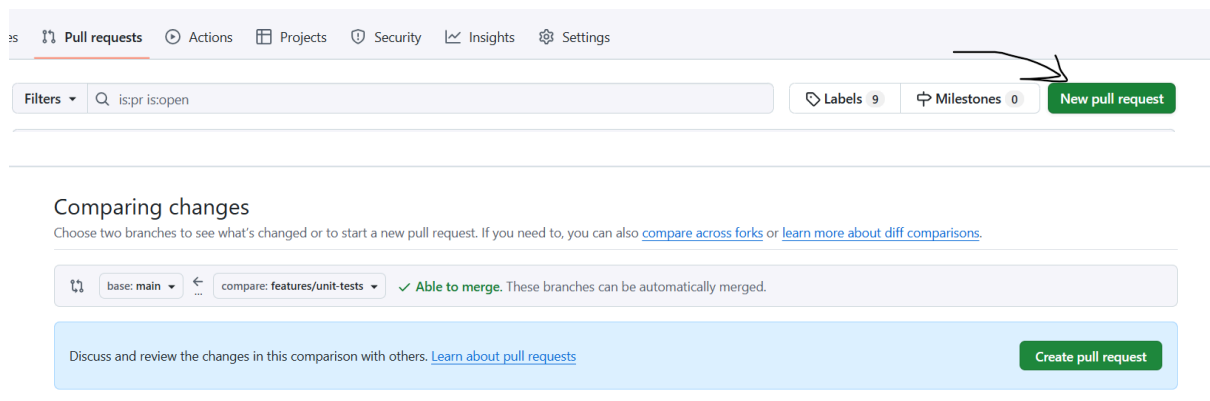
Створення коміту.



Натиснувши кнопку Commit лише створюються коміт, для того, щоб одразу завантажити зміни у віддалений репозиторій потрібно натиснути Commit and Push. Також можна завантажувати зміни окремо.



Створення Pull Request.



Create a new pull request by comparing changes across two branches. If you need to, you can also [compare across forks](#). [Learn more about diff comparisons here](#).

Remember, contributions to this repository should follow our [GitHub Community Guidelines](#).

Опис написання юніт-тестів.

Тестувалася програмна реалізація операцій над матрицями, згаданими вище. Загальна структура тестів така, що спочатку тестується базовий випадок з всіма коректними даними, потім граничний випадок з пустими матрицями, потім граничний випадок, з не коректними вхідними даними, для яких коректних вихідних даних не може бути за визначенням (наприклад знаходження оберненої матриці для сингулярної).

Лабораторна робота виконана мовою Python, тести я написала за допомогою модуля unittest.

Створюється клас, що успадковується від `unittest.TestCase`.

У класі реалізовано метод `setUp()`, що створює задані в ньому матриці перед кожним тестом, для визначення прикладів матриць та чисел, з якими проводиться тестування. Потрібно для зручності і уникнення повторення коду. Визначено дві прямокутні матриці однакової розмірності 2×3 , що потрібно для тестування додавання та віднімання, одиничну матриць розмірності 3, нульову матрицю розмірності 3, дві квадратні

матриці, одна з яких інвертована, а інші сингулярна, ціле число = 3 та число з плаваючою комою = $\frac{1}{2}$. Даний набір покриває і інші функції, оскільки можна помножити одну з матриць для додавання та будь-яку квадратну матрицю розмірності три (один з прикладів).

У даній лабораторній роботі я реалізувала 33 юніт-тести, що були пройдені успішно.

Опис тестів:

- Тестування додавання

1. `test_add()`: Перевіряє коректність додавання двох матриць однакових розмірів.
2. `test_add_empty()`: Перевіряє, чи виникає помилка `ValueError` при спробі додати порожні матриці.
3. `test_add_not_same()`: Перевіряє, чи виникає помилка `ValueError` при спробі додати матриці різних розмірів.

- Тестування віднімання

1. `test_sub()`: Перевіряє коректність віднімання однієї матриці від іншої.
2. `test_sub_empty()`: Перевіряє, чи виникає помилка `ValueError` при спробі відняти порожні матриці.
3. `test_sub_not_same()`: Перевіряє, чи виникає помилка `ValueError` при спробі відняти матриці різних розмірів.

- Тестування роботи функції множення на скаляр.

1. `test_mul_scalar()`: Перевіряє коректність множення матриці на скаляр (на число).
2. `test_mul_scalar_zero()`: Перевіряє, чи множення на нуль-скаляр або нуль-матрицю дає нуль-матрицю.
3. `test_mul_scalar_empty()`: Перевіряє, чи виникає помилка `ValueError` при множенні порожньої матриці на скаляр.

- Тестування множення двох матриць

1. `test_mul()`: Перевіряє коректність множення двох матриць (включаючи множення на одиничну матрицю).
2. `test_mul_empty()`: Перевіряє, чи виникає помилка `ValueError` при множенні порожніх матриць.
3. `test_mul_not_appropriate()`: Перевіряє, чи виникає помилка `ValueError` при множенні матриць з невідповідними розмірами.

- Тестування транспонування.

1. `test_transpose()`: Перевіряє коректність операції транспонування матриці.
2. `test_transpose_empty()`: Перевіряє, чи виникає помилка `ValueError` при транспонуванні порожньої матриці.

- Тестування елементарних перетворень рядків. (`scale_rows` потрібна для зручності в Гаусових перетвореннях для рангу, визначника та знаходження оберненої матриці)

1. `test_swap()`: Перевіряє, чи коректно міняються місцями два рядки матриці.
2. `test_swap_empty()`: Перевіряє, чи виникає помилка `ValueError` при зміні рядків у порожній матриці.
3. `test_swap_not_exists()`: Перевіряє, чи виникає помилка `IndexError` при спробі звернутися до неіснуючого рядка.
4. `test_scale()`: Перевіряє коректність множення рядка матриці на скаляр (масштабування).
5. `test_scale_empty()`: Перевіряє, чи виникає помилка `ValueError` при масштабуванні рядка в порожній матриці.

6. `test_scale_zero()`: Перевіряє, чи виникає помилка `ValueError` при масштабуванні рядка на нульовий фактор.
7. `test_scale_not_exists()`: Перевіряє, чи виникає помилка `IndexError` при масштабуванні неіснуючого рядка.
8. `test_add_rows()`: Перевіряє коректність додавання одного рядка до іншого (з фактором і без).
9. `test_add_rows_empty()`: Перевіряє, чи виникає помилка `ValueError` при додаванні рядків у порожній матриці.
10. `test_add_rows_zero_factor()`: Перевіряє, чи виникає помилка `ValueError` при додаванні рядка з нульовим фактором.
11. `test_add_rows_not_exists()`: Перевіряє, чи виникає помилка `IndexError` при додаванні неіснуючих рядків.

- Тестування обчислення визначника

1. `test_det()`: Перевіряє коректність обчислення визначника для звичайної та одиничної матриць.
2. `test_det_zero()`: Перевіряє, чи визначник сингулярної (виродженої) матриці коректно дорівнює 0.
3. `test_det_not_exists()`: Перевіряє, чи виникає помилка `ValueError` при обчисленні визначника для неквадратної матриці.

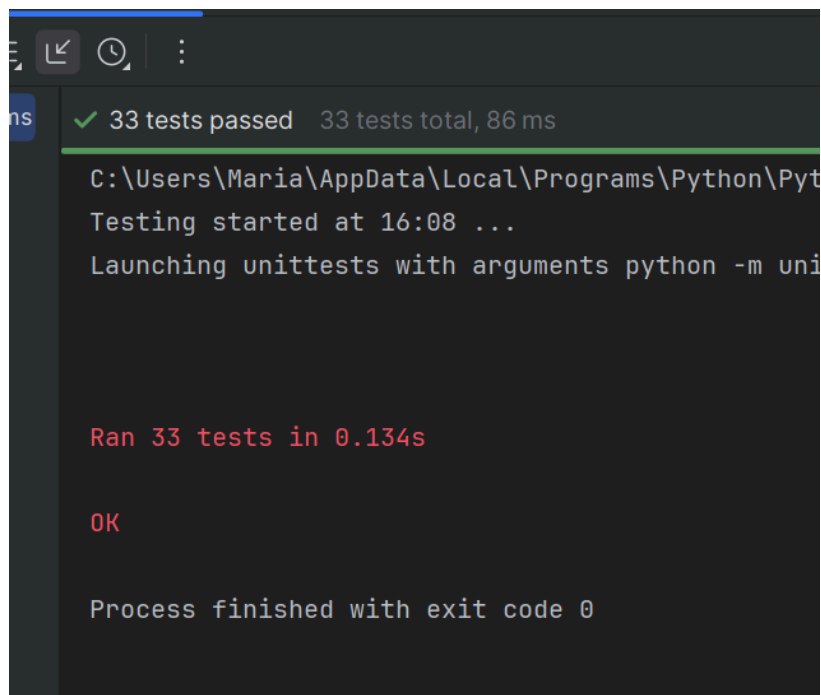
- Тестування роботи функції для знаходження оберненої матриці.

1. `test_inverse()`: Перевіряє коректність обчислення оберненої матриці.
2. `test_inverse_singular()`: Перевіряє, чи виникає помилка `ValueError` при спробі знайти обернену матрицю для сингулярної (виродженої).
3. `test_inverse_not_square()`: Перевіряє, чи виникає помилка `ValueError` при спробі знайти обернену матрицю для неквадратної.

- Тестування обчислення рангу.

1. `test_rank()`: Перевіряє коректність обчислення рангу для звичайної та сингулярної матриць.
2. `test_rank_empty()`: Перевіряє, чи виникає помилка `ValueError` при обчисленні рангу порожньої матриці.

Результат запуску тестів.

A screenshot of a terminal window with a dark background. At the top, a green bar displays '✓ 33 tests passed 33 tests total, 86 ms'. Below this, the terminal shows the file path 'C:\Users\Maria\AppData\Local\Programs\Python\Python38\Scripts\python.exe', the start time 'Testing started at 16:08 ...', and the command 'Launching unittests with arguments python -m unittest discover'. The main output is 'Ran 33 tests in 0.134s' in red text, followed by 'OK' in red. At the bottom, it says 'Process finished with exit code 0'.

Висновки про набуті навички.

У результаті виконання лабораторної роботи я здобула практичні навички роботи з системою контролю версій Git та платформою GitHub, освоївши ключові функції, такі як гілкування та `add`, `commit`, `push`, `merge`. Зокрема, я навчилася вести розробку в окремій гілці, що є важливою практикою для роботи над ізольованими завданнями, в моєму випадку — над юніт-тестуванням. У ході роботи я розробила структуру тестів та реалізувала набір тестових випадків, що покривають як базову функціональність, так і граничні умови. Це дозволило здобути навички виявлення та

виправлення помилок у вихідному коді на основі результатів, отриманих під час запуску тестів. Протягом усього процесу в репозиторії історія комітів. Завершальним етапом стало створення Pull Request, де було детально описано виконану роботу.